CSE 210: Computer Architecture Sessional
4-bit ALU Design and Implementation Report

Group Members:
Md. Fuad Al Alam (2205167)
Abdullah Siham Rahman (2205169)
Ali Akib Al-amin (2205170)
Fahim Faiaz Adib (2205177)
Gazi Faiaz Zafor Niloy (2205179)
Avrajit Biswas (2105126)

1 Introduction

An Arithmetic Logic Unit (ALU) is one of the core components of a computer system, responsible for performing arithmetic and logical operations on binary data. In this assignment for the course **CSE 210 (Computer Architecture Sessional)**, we were required to design and implement a **4-bit ALU** both in software simulation and in hardware using IC-level components.

The objective of this assignment was to gain hands-on experience in digital circuit design, understand the internal working of an ALU, and implement control-based operation selection efficiently while minimizing the number of ICs used. Our group (Section C, Subsection C2, Group 6) successfully designed, simulated, and implemented the assigned ALU operations along with the required status flags, following the given specifications, alhamdulillah.

2 Problem Specification

The functional design specification assigned to our group corresponds to **Section C2, Group 6**. The ALU operates on two 4-bit inputs A and B and is controlled using three control signals $cs2$, $cs1$, and $cs0$.

2.1 Arithmetic Operations

$cs2$	$cs1$	$cs0$	Operation
0	1	0	Add
0	1	1	Increment A
1	1	0	Sub with borrow
1	1	1	Transfer A

2.2 Logical Operations

$cs2$	$cs1$	$cs0$	Operation
X	0	0	OR
X	0	1	XOR

The ALU also generates the following status flags:

- Carry (C)
- Sign (S)
- Zero (Z)
- Overflow (V)

All flags were implemented following conventional ALU behavior as specified in the assignment.

3 Design Approach and Methodology

The ALU was designed following the conventional architecture in which the **arithmetic unit and logical unit operate in parallel**, and a final multiplexer selects the desired output. A 74HC157 multiplexer was used at the final stage to choose between the arithmetic and logical outputs.

It was observed that whenever $cs1 = 0$, the ALU performs logical operations (OR and XOR), and whenever $cs1 = 1$, it performs arithmetic operations. Therefore, $cs1$ determines the mode of operation and was directly connected to the select pin of the final multiplexer, allowing it to function as the **mode selection bit**.

3.1 Logical Unit Design

The logical unit was implemented using a simple and efficient three-IC design. A 74HC32 IC was used to perform the OR operation, and a 74HC86 IC was used to perform the XOR operation. The outputs of these two ICs were connected to a 74HC157 multiplexer, which selects the desired logical output.

The control signal $cs0$ was used as the select input of this multiplexer. When $cs0 = 0$, the OR operation is selected, and when $cs0 = 1$, the XOR operation is selected, producing the required logical output.

3.2 Arithmetic Unit Design

The arithmetic unit was implemented using a single 4-bit adder (74HC283) along with supporting control logic. The first input of the adder was connected directly to operand A. The second input was taken from a multiplexer that selected between operand B and its 1's complement ($\overline{B}$).

The 1's complement of B was generated using a hex inverter (74HC04). This configuration allowed both addition and subtraction to be implemented using the same adder.

Subtraction with borrow was implemented by adding the 1's complement of B to A without supplying a carry-in of 1, unlike conventional two's complement subtraction.

It was observed that whenever $cs0 = 1$, the required operations were **Increment A** and **Transfer A**, both of which require the second input of the adder to be zero. To achieve this without using additional ICs, the strobe (pin 15) of the multiplexer selecting B or $\overline{B}$ was used. By directly connecting $cs0$ to the active-low enable (strobe) pin of this multiplexer, the second input to the adder was disabled whenever $cs0 = 1$. This optimization significantly reduced the IC count.

The control signal $cs2$ was used as the select input of this multiplexer to correctly choose between addition and subtraction with borrow.

3.3 Carry-In Logic

The carry-in of the adder needed to be set to logic 1 only during the **Increment A** operation. This condition occurs when $cs2 = 0$ and $cs0 = 1$. Therefore, the carry-in was generated using the following logic:

$$C_{in} = \overline{cs2} \wedge cs0$$

This ensured that the carry-in was asserted only in the required mode.

3.4 Status Flag Design

All status flags were made as they would be in a real ALU. The Carry flag was not directly connected to the carry out bit of the adder or C_4, because it has to be active only when the ALU is in arithmetic mode. Hence it was passed through and AND gate with $cs1$ to ensure it only turns on when it's in arithmetic mode.

The Sign flag was generated directly from the most significant bit of the output (F_4).

The Zero flag was computed by ORing all output bits together and passing the result through a NOT gate, effectively implementing a 4-input NOR gate.

Overflow detection was implemented using the expression:

$$V = (X_4 \odot Y_4) \wedge (X_4 \oplus S_4)$$

Where X is the first input of the 4 bit adder (which is equivalent to input A in our ALU), Y is the second input (which comes from the MUX that selects between B and $\overline{B}$) and S is the output of the adder.

This expression is equivalent to $C_4 \oplus C_3$ and was used because internal access to C_3 was not available. We can get a simple derivation of the formula from the fact that whenever there's an overflow, it's either of two cases. One case is that the two inputs are positive (in their sign bit) but give visibly negative output (sign bit becomes 1) and another case is that the two inputs are negative (in their sign bit) but give visibly positive output (sign bit becomes 0). In both cases, numbers of same sign are added which is checked by $X_4 \odot Y_4$, and the sign of the output is opposite of the sign of inputs, which is checked by the other multiplicand $(X_4 \oplus S_4)$. Hence both happening at once indicates there'd be an overflow which is computed by passing them through an AND gate. However we also need to remember that overflow can only occur in arithmetic mode, hence we pass it again through another AND gate with $cs1$ like the case of Carry flag.

This completes the main ALU design methodology.

4 Truth Table

Since our design was very simple and our specifications were very easy, we needed no remapping of the control bits alhamdulillah. The specific inputs that were given to the adder are:

$cs2$	$cs1$	$cs0$	Operation	Adder Input 1	Adder Input 2	Carry In
0	1	0	Add	A	B	0
0	1	1	Increment A	A	0000	1
1	1	0	Sub with borrow	A	$\overline{B}$	0
1	1	1	Transfer A	A	0000	0

5 Block Diagram

To be drawn by me

6 Complete Circuit Diagram

The printed circuit design

7 ICs Used

IC Number	Description	Quantity
74HC04	Hex Inverter	1
74HC08	Quad AND Gate	1
74HC32	Quad OR Gate	2
74HC86	Quad XOR Gate	2
74HC157	Quad 2-to-1 Multiplexer	3
74HC283	4-bit Adder	1

8 Simulation Tool

- Software Used: Logisim Evolution
- Version: 4.0.0

9 Discussion

Alhamdulillah, the overall design and implementation process proceeded smoothly. The primary design challenge involved optimizing IC usage for Transfer A and Increment A operations, which initially required additional logic. This issue was resolved by using the strobe pin of the multiplexer to disable input B, thereby reducing IC count.

During hardware implementation, several practical issues were encountered. Some LEDs were damaged, causing faulty input or output indications, while others required higher voltage to light up. One LED was connected in reverse orientation, resulting in no output until corrected. Additionally, the carry-in of the adder was initially fixed at logic 1 during Transfer A and Increment A operations, which had to be identified and fixed.

Wire management also posed challenges, as dense wiring made debugging difficult. This was addressed by routing older wires closer to ICs and placing newer wires further away. A combination of flat rectangular wires and flexible jumper wires was used to balance cleanliness and flexibility.

Overall, this project provided valuable hands-on experience in ALU design, optimization, debugging, and practical hardware implementation.